

Speaker Accent Recognition

IEEE Publication Technology Department

I. DESCRIPCIÓN DEL PROBLEMA

EL reconocimiento automático de la voz ha avanzado significativamente en la última década, permitiendo interactuar de una manera más natural con distintos dispositivos electrónicos. Sin embargo, para lograr identificar lo que una persona está diciendo, es necesario enfrentarse al fenómeno lingüístico que representa el acento. Los acentos se pueden ver como patrones de entonación y ritmos que son propios de una región, pero el concepto abarca mucho más que solo eso.

El acento constituye una huella sonora característica en el habla de individuos pertenecientes a diferentes zonas geográficas. Esta variabilidad en el habla plantea desafíos para los sistemas de reconocimiento de la voz. Algunos ejemplos de esto son los sistemas como Siri, Alexa o el Google Assistant, los cuales son diseñados para interpretar comandos a través de la voz y ejecutarlos. En este ejemplo, el sistema debe reconocer muy bien a qué se refiere el usuario independientemente del acento que este posea al hablar.

II. EXPLICACIÓN DEL DATASET

La base de datos contiene 329 muestras de audio de 1 segundo donde hablantes de seis países (ES, FR, GE, IT, UK, US) pronuncian una palabra en inglés. A cada audio se le aplicó la técnica MFCC, obteniendo 12 características numéricas por muestra ($X_1, X_2, \dots, X_{12}$).

Para convertir un archivo de audio a las 12 variables numéricas mencionadas anteriormente, los autores del dataset aplicaron el algoritmo descrito en estos 5 pasos.

1. Preprocesamiento: Tomar la frecuencia y aplicarle una transformación de manera que se realcen las frecuencias altas. De esta manera la amplitud tanto de las frecuencias altas como de las bajas se vuelven aproximadamente similares y por tanto, son comparables.
2. Se divide la señal utilizando MFCC en la banda sonora del dominio de tiempo original de 1s máximo de lectura de una palabra. A cada fragmento se le aplica la transformada rápida de fourier, de esta manera se logra obtener el espectro de frecuencia
3. Se aplican varios filtros en escala de Mel. Esto agrupa las frecuencias en bandas que imitan la percepción humana
4. Finalmente se aplica la transformada discreta del coseno para obtener los 12 coeficientes

En cuanto a la fase de caracterización, hallamos que el dataset no cuenta con datos faltantes, por lo que no fue necesario realizar imputación; no obstante, en el conteo de muestras por clase evidenciamos un aparente desbalance:

Lo cual nos plantea la pregunta de si nos enfrentamos a un problema de clases desbalanceadas, ya que en la Tabla 1, podemos ver que hay una predominancia de muestras para

Lenguaje	Total Muestras
ES	29
FR	30
GE	30
IT	30
UK	30
US	165

Cuadro I

DISTRIBUCIÓN DE MUESTRAS POR ACENTO EN EL DATASET.

la clase "US". Si bien no hay un estándar único que nos permitiera definir un desbalance en los datos, hay umbrales ampliamente aceptados en la comunidad de ciencia de datos. En nuestro caso, aplicamos el "Imbalance Ratio" definido como: "El imbalance ratio (IR) se define como la relación entre el número de muestras de clase mayoritaria y el número de muestras de clase minoritaria" [1]. En la práctica, se considera que un conjunto de datos está desbalanceado si el IR es superior a 3, es decir, cuando la clase mayoritaria contiene al menos tres veces más ejemplos que la clase minoritaria. Para nuestro caso, obtuvimos un IR de 5.6, lo cual nos permite concluir que estamos ante un problema de este tipo.

En cuanto a la distribución y rango de las características MFCC, sin distinción por lenguaje encontramos que las características tienen rangos muy diferentes (Tabla 2), cabe evaluar qué tan diferentes deben ser para considerar aplicar un criterio de normalización o estandarización, puesto que si no se trata a tiempo podría convertirse en un problema en la aplicación de modelos basados en distancias, ya que las características con rangos más grandes podrían dominar la función de distancias o los cálculos del gradiente.

X	Min	Max	Rango
1	-6.06	17.75	23.8
2	-14.97	3.57	18.5
7	-15.37	-0.42	14.9
1	-15.51	4.79	20.3

Cuadro II

ELABORACIÓN PROPIA.

En relación con la distribución promedio de MFCCs por lenguaje, en la gráfica 1 podemos ver que hay suficiente variabilidad y sutiles diferencias en los MFCCs que sugieren que pueden ser características valiosas para entrenar un modelo de detección de acento. El siguiente paso crucial es analizar si estas diferencias son estadísticamente significativas entre los diferentes acentos.

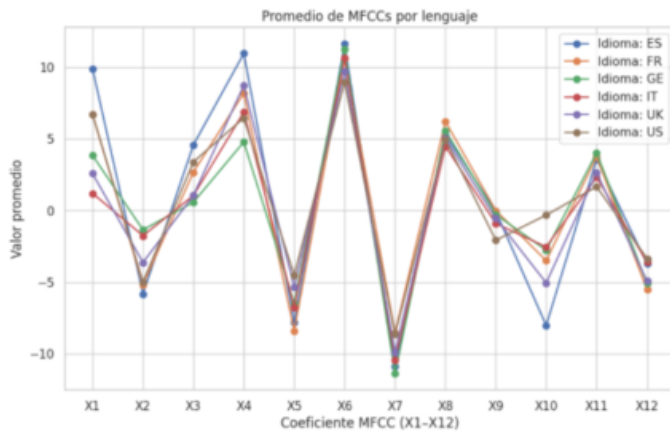


Figura 1. Descripción de la imagen.

III. ESTADO DEL ARTE

En [1], los autores evalúan nueve modelos de aprendizaje automático: K-Nearest Neighbors (KNN), Perceptrón Multicapa, Red de Funciones de Base Radial, Árboles de Decisión, Árbol Logístico, Bosques Aleatorios (Random Forest), K-Star, Regresión Logística y Naive Bayes, siendo el KNN y el Multilayer Perceptron (MLP) los modelos con mejores resultados. Para la validación de los modelos, emplearon el método de validación cruzada mediante K-Folds con 3 folds.

Las métricas usadas fueron exactitud, sensibilidad, F-score y kappa statistics, con el fin de obtener una evaluación más completa del desempeño de los modelos.

El kappa statistics o coeficiente kappa de Cohen es una métrica usada en los problemas de clasificación multiclase y especialmente útil en problemas ordinales. Es utilizada porque los criterios como accuracy, exactitud y sensibilidad no muestran completamente el rendimiento del modelo, especialmente cuando existe un desbalance entre las clases. El coeficiente kappa tiene en cuenta los aciertos que se esperarían por el azar, proporcionando así una medida más confiable sobre la concordancia entre las predicciones del modelo y las verdaderas etiquetas.

Wenjuan Ke propone un modelo de clasificación de acentos en inglés basado en deep learning, combinando características MFCC y FBank con una arquitectura de red 1D CNN-BiGRU con mecanismo de atención [2]. El sistema fue entrenado y evaluado con el corpus VoxForge, utilizando seis acentos (AM, BR, EU, CA, IN, AU). La arquitectura final logró una F1 score promedio de 85.52 %, con valores superiores al 80 % en todas las clases. También se evidenció que FBank ofreció mejores resultados que MFCC, y que el uso de atención y redes bidireccionales (BiGRU) mejora significativamente el desempeño frente a variantes más simples.

En el artículo titulado “A Comparison of Classifiers in Performing Speaker Accent Recognition Using MFCCs” [3] se utiliza un paradigma de aprendizaje supervisado para resolver el problema de reconocimiento de acentos mediante

la clasificación binaria (acento US vs. no-US). Para ello, los autores emplean técnicas de aprendizaje como análisis discriminante (LDA y QDA), máquinas de vectores de soporte (SVM con kernel RBF y polinomial) y k-vecinos más cercanos (K-NN), utilizando vectores promedio MFCCs como características. La metodología de validación usada fue una validación cruzada tipo holdout con muestreo aleatorio estratificado y tamaño de muestra de 500 repeticiones. La métrica principal utilizada para evaluar el desempeño fue la exactitud (accuracy). En cuanto a resultados, el clasificador K-NN obtuvo el mejor desempeño, alcanzando una exactitud promedio de 0.9605 con 39 MFCCs, superando a SVM y QDA tanto en exactitud como en tiempo de cómputo, lo que demuestra su eficacia para esta tarea.

En [4], los autores proponen un sistema de clasificación de acento para hablantes de inglés americano, usando aprendizaje supervisado con modelos ocultos de Markov (HMM). Se evaluaron tres configuraciones de modelado acústico: IW-FS, con modelos por palabra aplicados a habla libre; CS-FS, con modelos fonéticos contextuales aplicados también a habla libre; y CS-PS, que usa los mismos modelos fonéticos pero evaluados con texto conocido, lo que reduce la variabilidad del habla.

Las características utilizadas fueron MFCC, energía y sus derivadas. Se aplicaron pruebas abiertas con hablantes no vistos en el entrenamiento. Las métricas principales fueron las tasas de clasificación (asigna una etiqueta de acento predefinida a una muestra de voz) y detección de acento (Determina si hay un acento o no). El mejor desempeño se logró con IW-FS, alcanzando un 93 % de exactitud con secuencias de 7–8 palabras. También mostraron que usar modelos específicos por acento puede reducir el error en el reconocimiento de voz hasta en un 73 %.

IV. METODOLOGÍA DE VALIDACIÓN

Se utilizó validación cruzada estratificada con 5 particiones ($k=5$) para entrenar y evaluar los modelos de Machine Learning, ya que esta técnica mantiene la proporción de clases en cada conjunto, lo cual es clave debido al desbalance de clases en el dataset. Esto permite obtener estimaciones más precisas del rendimiento del modelo.

IV-A. Etapa adicional

Para reducir el impacto del desbalance de clases durante el entrenamiento, se utilizó la técnica de sobremuestreo SMOTE, aplicada solo al conjunto de entrenamiento dentro de cada fold. SMOTE genera ejemplos sintéticos de las clases minoritarias, equilibrando los datos antes de entrenar. La evaluación se realizó sobre los datos de validación originales (sin SMOTE) para evitar fuga de datos y asegurar una estimación realista del rendimiento del modelo.

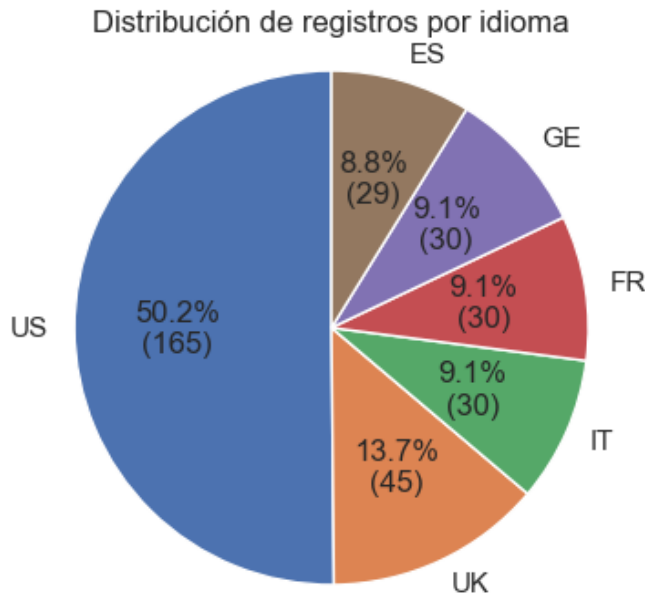


Figura 2. Distribución de registros por idioma en el dataset. Se observa un claro desbalance, con una predominancia significativa de muestras del idioma inglés americano (US).

V. CONJUNTO DE HIPERPARÁMETROS

Para la clasificación de acentos, se seleccionaron 5 modelos de Machine Learning que representan diferentes enfoques de modelado. La siguiente tabla resume los hiperparámetros explorados para cada modelo mediante búsqueda en cuadrícula (Grid Search):

Modelo	Hiperparámetros
Reg. Log.	C: [0.001-100], solver: ['liblinear', 'lbfgs'], penalty: ['l2']
KNN	n_neighbors: [3,5,7,9], weights: ['uniform', 'distance'], p: [1,2]
Random Forest	n_estimators: [50,100,200], max_depth: [None,10,20], min_samples_split: [2,5,10], min_samples_leaf: [1,2,4]
MLP	hidden_layer_sizes: [(50),(100),(50,50),(100,50)], activation: ['tanh','relu'], solver: ['adam','sgd'], alpha: [0.0001-0.01]
SVC	C: [0.1,1,10], kernel: ['linear','rbf'], gamma: ['scale','auto']

Cuadro III
HIPERPARÁMETROS EXPLORADOS PARA CADA MODELO

VI. MÉTRICAS DE DESEMPEÑO

Debido al desbalance de las clases, se emplearon métricas más adecuadas que el accuracy, para evaluar el rendimiento del modelo. Las métricas seleccionadas fueron:

1. **Matriz de Confusión:** Permite ver si el modelo confunde ciertos acentos y cómo maneja las clases con menos datos.
2. **Precisión por clase y macro-promedio:** Permite identificar si cuando el modelo predice un acento, realmente es ese acento. Es útil para evitar falsos positivos.

Recall por clase y macro-promedio: Nos dice si el modelo es bueno encontrando todas las instancias de cada acento, importante para no dejar pasar acentos minoritarios.

Puntuación F1 (F1-score) por clase y macro-promedio: Combina precisión y recall, siendo especialmente útil en datasets desbalanceados para evaluar el rendimiento global de forma equilibrada.

Nota: Usamos el average macro porque usaremos SMOTE y por tanto las clases estarán balanceadas. Si estuvieran desbalanceadas como inicialmente se plantea el problema usaríamos el average weighted que tiene en cuenta el número de muestras de la clase.

VII. RESULTADOS DE EXPERIMENTACIÓN

Se presentan los resultados de la validación cruzada estratificada con SMOTE para cada modelo y combinación de hiperparámetros. Las tablas reportan el promedio y la desviación estándar de las métricas macro (F1, Precisión y Recall) calculadas en los 5 folds. La desviación estándar muestra la consistencia del modelo: valores bajos indican un rendimiento más estable entre folds.

VII-A. Resultados de la validación cruzada para cada modelo

VII-A1. Regresión Logística: Mejor combinación de hiperparámetros encontrada para Regresión Logística (basado en f1_macro promedio):

Hiperparámetro	Valor
C	1
penalty	'l2'
solver	'lbfgs'
F1-macro promedio	0.6617 ± 0.0313

Cuadro IV

MEJORES HIPERPARÁMETROS PARA REGRESIÓN LOGÍSTICA

	Hyperparameters	f1_macro	precision_macro	recall_macro
0	{C: 0.001, 'penalty': 'l2', 'solver': 'liblin...	0.5285 ± 0.0597	0.5518 ± 0.0597	0.6380 ± 0.0382
1	{C: 0.001, 'penalty': 'l2', 'solver': 'lbfgs'}	0.5941 ± 0.0397	0.5834 ± 0.0431	0.6877 ± 0.0278
2	{C: 0.01, 'penalty': 'l2', 'solver': 'liblin...	0.5883 ± 0.0370	0.5768 ± 0.0351	0.6746 ± 0.0354
3	{C: 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}	0.6295 ± 0.0220	0.6066 ± 0.0188	0.7031 ± 0.0242
4	{C: 0.1, 'penalty': 'l2', 'solver': 'libline...	0.5977 ± 0.0422	0.5812 ± 0.0429	0.6866 ± 0.0357
5	{C: 0.1, 'penalty': 'l2', 'solver': 'lbfgs'}	0.6570 ± 0.0380	0.6415 ± 0.0337	0.7201 ± 0.0418
6	{C: 1, 'penalty': 'l2', 'solver': 'liblinear'}	0.6129 ± 0.0418	0.5956 ± 0.0437	0.6979 ± 0.0380
7	{C: 1, 'penalty': 'l2', 'solver': 'lbfgs'}	0.6617 ± 0.0313	0.6536 ± 0.0420	0.7203 ± 0.0154
8	{C: 10, 'penalty': 'l2', 'solver': 'liblinear'}	0.6403 ± 0.0228	0.6247 ± 0.0262	0.7107 ± 0.0293
9	{C: 10, 'penalty': 'l2', 'solver': 'lbfgs'}	0.6491 ± 0.0324	0.6392 ± 0.0413	0.7081 ± 0.0141
10	{C: 100, 'penalty': 'l2', 'solver': 'libline...	0.6369 ± 0.0233	0.6210 ± 0.0298	0.7023 ± 0.0246
11	{C: 100, 'penalty': 'l2', 'solver': 'lbfgs'}	0.6354 ± 0.0383	0.6317 ± 0.0428	0.6895 ± 0.0357

Figura 3. Resultados de validación cruzada para el modelo de Regresión Logística, mostrando el desempeño para diferentes combinaciones de hiperparámetros.

VII-A2. K-Neighbors Classifier: Mejor combinación de hiperparámetros encontrada para K-Neighbors Classifier (basado en f1_macro promedio):

Hiperparámetro	Valor
n_neighbors	3
p	2
weights	'distance'
F1-macro promedio	0.7830 ± 0.0192

Cuadro V

MEJORES HIPERPARÁMETROS PARA K-NEIGHBORS

	Hyperparameters	f1_macro	precision_macro	recall_macro
0	{'n_neighbors': 3, 'p': 1, 'weights': 'uniform'}	0.7734 ± 0.0161	0.7541 ± 0.0074	0.8438 ± 0.0148
1	{'n_neighbors': 3, 'p': 1, 'weights': 'distance'}	0.7771 ± 0.0175	0.7606 ± 0.0248	0.8402 ± 0.0164
2	{'n_neighbors': 3, 'p': 2, 'weights': 'uniform'}	0.7764 ± 0.0101	0.7560 ± 0.0154	0.8451 ± 0.0070
3	{'n_neighbors': 3, 'p': 2, 'weights': 'distance'}	0.7830 ± 0.0192	0.7642 ± 0.0217	0.8426 ± 0.0145
4	{'n_neighbors': 5, 'p': 1, 'weights': 'uniform'}	0.7424 ± 0.0380	0.7253 ± 0.0183	0.8296 ± 0.0402
5	{'n_neighbors': 5, 'p': 1, 'weights': 'distance'}	0.7493 ± 0.0325	0.7301 ± 0.0170	0.8291 ± 0.0382
6	{'n_neighbors': 5, 'p': 2, 'weights': 'uniform'}	0.7432 ± 0.0300	0.7232 ± 0.0217	0.8327 ± 0.0273
7	{'n_neighbors': 5, 'p': 2, 'weights': 'distance'}	0.7641 ± 0.0251	0.7431 ± 0.0286	0.8453 ± 0.0258
8	{'n_neighbors': 7, 'p': 1, 'weights': 'uniform'}	0.7285 ± 0.0382	0.7061 ± 0.0286	0.8194 ± 0.0353
9	{'n_neighbors': 7, 'p': 1, 'weights': 'distance'}	0.7464 ± 0.0531	0.7280 ± 0.0396	0.8298 ± 0.0455
10	{'n_neighbors': 7, 'p': 2, 'weights': 'uniform'}	0.7340 ± 0.0350	0.7281 ± 0.0182	0.8268 ± 0.0196
11	{'n_neighbors': 7, 'p': 2, 'weights': 'distance'}	0.7334 ± 0.0306	0.7245 ± 0.0224	0.8241 ± 0.0177
12	{'n_neighbors': 9, 'p': 1, 'weights': 'uniform'}	0.7061 ± 0.0285	0.6837 ± 0.0249	0.8024 ± 0.0264
13	{'n_neighbors': 9, 'p': 1, 'weights': 'distance'}	0.7189 ± 0.0574	0.6991 ± 0.0481	0.8089 ± 0.0525

Figura 4. Resultados de validación cruzada para el modelo K-Neighbors, mostrando el impacto de diferentes valores de n_neighbors y métricas de distancia.

VII-A3. Random Forest Classifier: Mejor combinación de hiperparámetros encontrada para Random Forest Classifier (basado en f1_macro promedio):

Hiperparámetro	Valor
max_depth	10
min_samples_leaf	1
min_samples_split	2
n_estimators	100
F1-macro promedio	0.8044 ± 0.0508

Cuadro VI

MEJORES HIPERPARÁMETROS PARA RANDOM FOREST

	Hyperparameters	f1_macro	precision_macro	recall_macro
0	{'max_depth': None, 'min_samples_leaf': 1, 'min...	0.7813 ± 0.0415	0.7915 ± 0.0450	0.7876 ± 0.0388
1	{'max_depth': None, 'min_samples_leaf': 1, 'mi...	0.7971 ± 0.0445	0.7970 ± 0.0393	0.8098 ± 0.0496
2	{'max_depth': None, 'min_samples_leaf': 1, 'mi...	0.7854 ± 0.0494	0.7985 ± 0.0415	0.7888 ± 0.0589
3	{'max_depth': None, 'min_samples_leaf': 1, 'mi...	0.7498 ± 0.0634	0.7678 ± 0.0336	0.7613 ± 0.0773
4	{'max_depth': None, 'min_samples_leaf': 1, 'mi...	0.7911 ± 0.0387	0.7969 ± 0.0331	0.8023 ± 0.0481
...	...	...	...	...
76	{'max_depth': 20, 'min_samples_leaf': 4, 'min...	0.7853 ± 0.0369	0.7791 ± 0.0299	0.8100 ± 0.0489
77	{'max_depth': 20, 'min_samples_leaf': 4, 'min...	0.7782 ± 0.0236	0.7689 ± 0.0243	0.8034 ± 0.0305
78	{'max_depth': 20, 'min_samples_leaf': 4, 'min...	0.7805 ± 0.0329	0.7731 ± 0.0455	0.8115 ± 0.0329
79	{'max_depth': 20, 'min_samples_leaf': 4, 'min...	0.7818 ± 0.0299	0.7766 ± 0.0294	0.8061 ± 0.0357
80	{'max_depth': 20, 'min_samples_leaf': 4, 'min...	0.7870 ± 0.0241	0.7703 ± 0.0238	0.8194 ± 0.0279

81 rows × 4 columns

Figura 5. Resultados de validación cruzada para Random Forest, ilustrando el efecto de diferentes profundidades de árbol y número de estimadores.

VII-A4. MLP Classifier: Mejor combinación de hiperparámetros encontrada para MLP Classifier (basado en f1_macro promedio):

Hiperparámetro	Valor
activation	'tanh'
alpha	0.0001
hidden_layer_sizes	(100, 50)
learning_rate_init	0.001
solver	'adam'
F1-macro promedio	0.8412 ± 0.0460

Cuadro VII

MEJORES HIPERPARÁMETROS PARA MLP

	Hyperparameters	f1_macro	precision_macro	recall_macro
15	{'activation': 'tanh', 'alpha': 0.001, 'hidden...	0.8090 ± 0.0716	0.8133 ± 0.0456	0.8268 ± 0.0822
16	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.7616 ± 0.0526	0.7766 ± 0.0417	0.7714 ± 0.0746
17	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.7204 ± 0.0481	0.7185 ± 0.0496	0.7578 ± 0.0638
18	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.8126 ± 0.0397	0.8197 ± 0.0237	0.8241 ± 0.0554
19	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.8179 ± 0.0500	0.8144 ± 0.0430	0.8367 ± 0.0587
20	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.7646 ± 0.0623	0.7785 ± 0.0751	0.7718 ± 0.0652
21	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.7466 ± 0.0518	0.7630 ± 0.0361	0.7568 ± 0.0690
22	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.8412 ± 0.0460	0.8594 ± 0.0428	0.8386 ± 0.0510
23	{'activation': 'tanh', 'alpha': 0.01, 'hidden...	0.8090 ± 0.0716	0.8133 ± 0.0456	0.8268 ± 0.0822
24	{'activation': 'relu', 'alpha': 0.0001, 'hidde...	0.7823 ± 0.0485	0.8078 ± 0.0300	0.7885 ± 0.0697
25	{'activation': 'relu', 'alpha': 0.0001, 'hidde...	0.8162 ± 0.0531	0.8234 ± 0.0432	0.8294 ± 0.0580
26	{'activation': 'relu', 'alpha': 0.0001, 'hidde...	0.8298 ± 0.0293	0.8474 ± 0.0410	0.8328 ± 0.0454
27	{'activation': 'relu', 'alpha': 0.0001, 'hidde...	0.8052 ± 0.0474	0.8108 ± 0.0358	0.8237 ± 0.0552
28	{'activation': 'relu', 'alpha': 0.0001, 'hidde...	0.7802 ± 0.0454	0.7887 ± 0.0495	0.7831 ± 0.0509

Figura 6. Resultados de validación cruzada para la Red Neuronal MLP, mostrando el desempeño con diferentes arquitecturas y parámetros de entrenamiento.

VII-A5. SVC: Mejor combinación de hiperparámetros encontrada para SVC (basado en f1_macro promedio):

Hiperparámetro	Valor
C	10
gamma	'scale'
kernel	'rbf'
F1-macro promedio	0.7900 ± 0.0412

Cuadro VIII

MEJORES HIPERPARÁMETROS PARA SVC

	Hyperparameters	f1_macro	precision_macro	recall_macro
0	{'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}	0.6592 ± 0.0278	0.6415 ± 0.0271	0.7230 ± 0.0310
1	{'C': 0.1, 'gamma': 'scale', 'kernel': 'rbf'}	0.4586 ± 0.0180	0.4923 ± 0.0215	0.5901 ± 0.0222
2	{'C': 0.1, 'gamma': 'auto', 'kernel': 'linear'}	0.6592 ± 0.0278	0.6415 ± 0.0271	0.7230 ± 0.0310
3	{'C': 0.1, 'gamma': 'auto', 'kernel': 'rbf'}	0.2834 ± 0.0274	0.6221 ± 0.1255	0.2675 ± 0.0162
4	{'C': 1, 'gamma': 'scale', 'kernel': 'linear'}	0.6599 ± 0.0489	0.6411 ± 0.0460	0.7214 ± 0.0447
5	{'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}	0.7146 ± 0.0194	0.6963 ± 0.0211	0.7896 ± 0.0246
6	{'C': 1, 'gamma': 'auto', 'kernel': 'linear'}	0.6599 ± 0.0489	0.6411 ± 0.0460	0.7214 ± 0.0447
7	{'C': 1, 'gamma': 'auto', 'kernel': 'rbf'}	0.4779 ± 0.0750	0.7802 ± 0.1165	0.4158 ± 0.0571
8	{'C': 10, 'gamma': 'scale', 'kernel': 'linear'}	0.6457 ± 0.0396	0.6298 ± 0.0424	0.7063 ± 0.0390
9	{'C': 10, 'gamma': 'scale', 'kernel': 'rbf'}	0.7900 ± 0.0412	0.7826 ± 0.0441	0.8330 ± 0.0473
10	{'C': 10, 'gamma': 'auto', 'kernel': 'linear'}	0.6457 ± 0.0396	0.6298 ± 0.0424	0.7063 ± 0.0390
11	{'C': 10, 'gamma': 'auto', 'kernel': 'rbf'}	0.5004 ± 0.0644	0.8038 ± 0.0783	0.4324 ± 0.0479

Figura 7. Resultados de validación cruzada para SVM, mostrando el impacto de diferentes kernels y parámetros de regularización.

VII-B. Resumen de los Mejores Resultados por Modelo en Validación Cruzada

En la siguiente tabla podemos observar el rendimiento promedio F1-macro y su desviación estándar para la mejor

combinación de hiperparámetros encontrada para cada uno de los modelos evaluados durante la validación cruzada estratificada con SMOTE.

Modelo	Mejores Hiperparámetros	F1-Macro
Reg. Log.	C=1, penalty='l2', solver='lbfgs'	0.662 $\pm$ 0.031
KNN	n_neighbors=3, p=2, weights='distance'	0.783 $\pm$ 0.019
Random Forest	max_depth=None, min_samples_leaf=1, n_estimators=100	0.804 $\pm$ 0.051
MLP	activation='tanh', alpha=0.0001, hidden=(100,50), solver='adam'	0.841 $\pm$ 0.046
SVC	C=10, gamma='scale', kernel='rbf'	0.790 $\pm$ 0.041

Cuadro IX
RESUMEN DE RESULTADOS POR MODELO

Podemos destacar que los mejores modelos fueron aquellos que maximizaron el f1-macro, los cuales son Random Forest y MLP.

VIII. ANÁLISIS INDIVIDUAL DE CARACTERÍSTICAS

Para hacer el análisis de selección de características empleamos la correlación de Pearson, en el cual si se tiene un coeficiente de correlación mayor a .7 entre dos o más predictores, esto indica multicolinealidad (redundancia entre variables independientes).

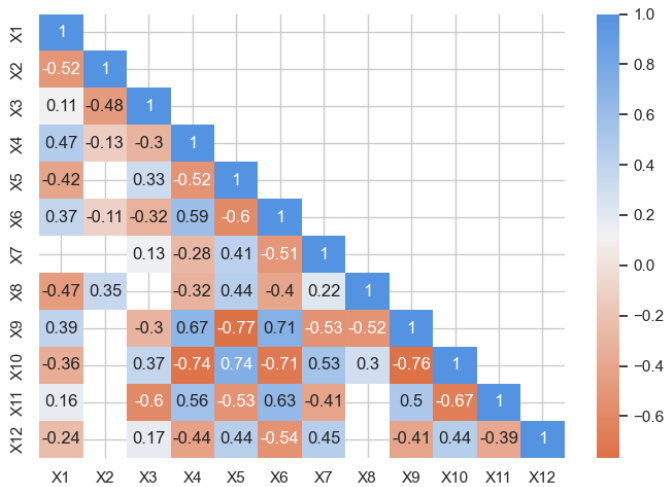


Figura 8. Mapa de calor de correlaciones de Pearson entre características, revelando las relaciones lineales entre variables.

El análisis de correlación ($|r| > 0.7$) reveló fuertes relaciones entre:

- X5 y X6 con X9
- X4, X5 y X6 con X10
- X9 con X10

Dado que X9 y X10 concentran la información de las demás, eliminarlas implicaría pérdida de información relevante. Por tanto, las variables X4, X5 y X6 se identifican como

candidatas a eliminar, ya que su contenido ya está representado por X9 y X10.

IX. SELECCIÓN DE CARACTERÍSTICAS

Por medio de un método de búsqueda ascendente, hallamos que el mejor subconjunto de características para Random Forest fue de: ('0', '1', '3', '4', '5', '6', '7', '9', '10', '11') Mejor puntuación de F1-macro (CV): 0.7462

Mejor subconjunto de características para MLP: ('0', '1', '2', '3', '4', '5', '6', '7', '9', '10', '11') Mejor puntuación de f1_macro (CV): 0.7504

Para Random Forest la característica descartada fue x3 y x9

Para MLP la característica descartada fue x9

Debido al fuerte desbalance de clases en el dataset (por ejemplo, la clase US tiene más de cinco veces más muestras que otras), se utilizó F1-weighted como función criterio en la selección de características. Esta métrica ajusta el impacto de cada clase según su frecuencia, equilibrando precisión y recall. Así se evita favorecer solo a la clase mayoritaria y se obtiene un subconjunto de características con mejor desempeño global entre clases.

X. REDUCCIÓN DE DIMENSIONALIDAD MEDIANTE ANÁLISIS DE COMPONENTES PRINCIPALES (PCA)

Antes de aplicar el análisis de componentes principales (PCA), se analizó la varianza de cada una de las variables originales, encontrando que algunas variables presentan varianzas muy superiores a otras, lo que indica una escala heterogénea y posibles redundancias de información. La gráfica de barras (Fig. 9) evidencia que las variables X1 y X10 tienen las mayores varianzas, mientras que variables como X7 y X8 aportan poca variabilidad al conjunto. Tras aplicar PCA y reducir las dimensiones a las primeras cinco componentes principales, se obtuvieron las nuevas variables PC1 a PC5, cuyas combinaciones lineales concentran la mayor parte de la información del conjunto original. La tabla (Fig. 11) muestra las coordenadas de las primeras observaciones proyectadas sobre estas componentes, ya en un espacio reducido. El gráfico de varianza explicada (Fig. 10) muestra que la primera componente por sí sola explica cerca del 45 % de la varianza total, y que acumulando hasta la quinta componente se alcanza aproximadamente un 85 % de la varianza, justificando la elección de las cinco componentes para representar adecuadamente los datos.

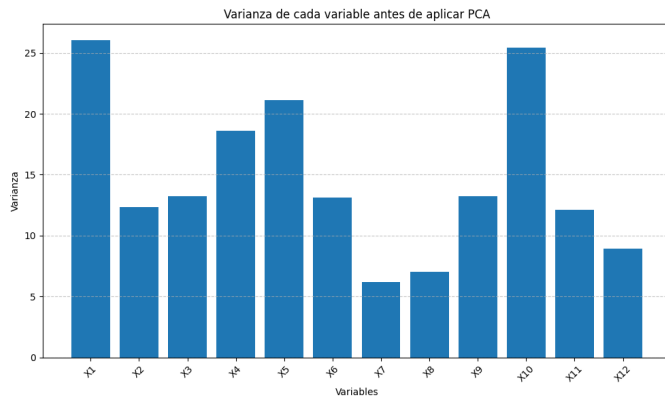


Figura 9. Varianza explicada por cada componente principal, mostrando la contribución individual de cada componente.

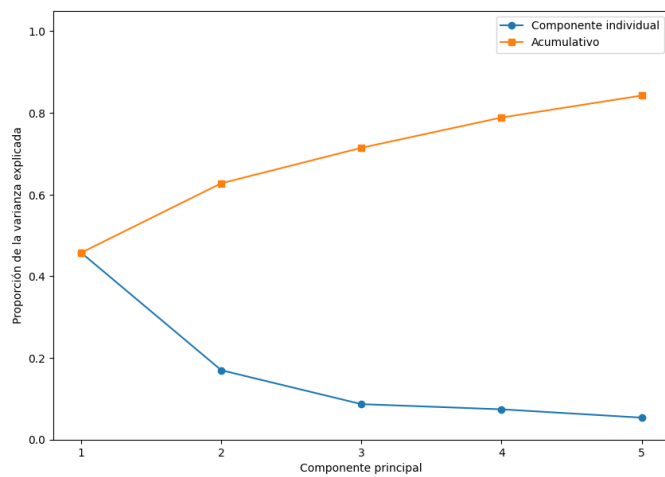


Figura 10. Varianza acumulada explicada por los componentes principales, ayudando a determinar el número óptimo de componentes.

PC1	PC2	PC3	PC4	PC5
-1.2267	-1.4061	-1.1865	-0.2768	-0.2840
-1.8160	-0.8731	-0.5867	0.0932	-0.4640
-1.1201	-0.9822	-2.0197	-0.0214	-0.3778
-0.9790	-0.5316	-1.6128	0.4331	-0.1750
-1.0998	-0.7168	-1.7083	0.0438	-0.3560

Figura 11. Tabla comparativa de resultados con diferentes números de componentes principales.

XI. COMPARACIÓN DE MODELOS Y SELECCIÓN DEL MEJOR MODELO

Se evaluaron cinco modelos (Regresión Logística, KNN, Random Forest, MLP Classifier y SVC) usando validación

cruzada estratificada con SMOTE para manejar el desbalance de clases, tomando como métrica principal el F1-macro promedio.

El MLP Classifier obtuvo el mejor rendimiento promedio con un F1-macro de 0.8412 ± 0.0460 , superando a los demás modelos. Esto indica que una red neuronal con la configuración óptima es la opción más adecuada para esta tarea multiclase con los datos disponibles. La desviación estándar moderada muestra que su desempeño es relativamente consistente entre las diferentes particiones del dataset.

MLP Classifier

Etiqueta	Precisión	Recall	F1-Score	Support
ES	0.5714	0.6667	0.6154	6.0000
FR	1.0000	0.5000	0.6667	6.0000
GE	0.6250	0.8333	0.7143	6.0000
IT	0.1667	0.1667	0.1667	6.0000
UK	0.4545	0.5556	0.5000	9.0000
US	0.7419	0.6970	0.7188	33.0000
Accuracy	0.6212	0.6212	0.6212	0.6212
Macro Avg	0.5933	0.5699	0.5636	66.0000
Weighted Avg	0.6478	0.6212	0.6242	66.0000

Cuadro X

RESULTADOS DETALLADOS DEL MLP CLASSIFIER

Random Forest Classifier

Etiqueta	Precisión	Recall	F1-Score	Support
ES	0.6667	0.6667	0.6667	6.0000
FR	0.7500	0.5000	0.6000	6.0000
GE	0.5455	1.0000	0.7059	6.0000
IT	0.5000	0.3333	0.4000	6.0000
UK	0.4167	0.5556	0.4762	9.0000
US	0.8276	0.7273	0.7742	33.0000
Accuracy	0.6667	0.6667	0.6667	0.6667
Macro Avg	0.6177	0.6305	0.6038	66.0000
Weighted Avg	0.6944	0.6667	0.6677	66.0000

Cuadro XI

RESULTADOS DETALLADOS DEL RANDOM FOREST CLASSIFIER

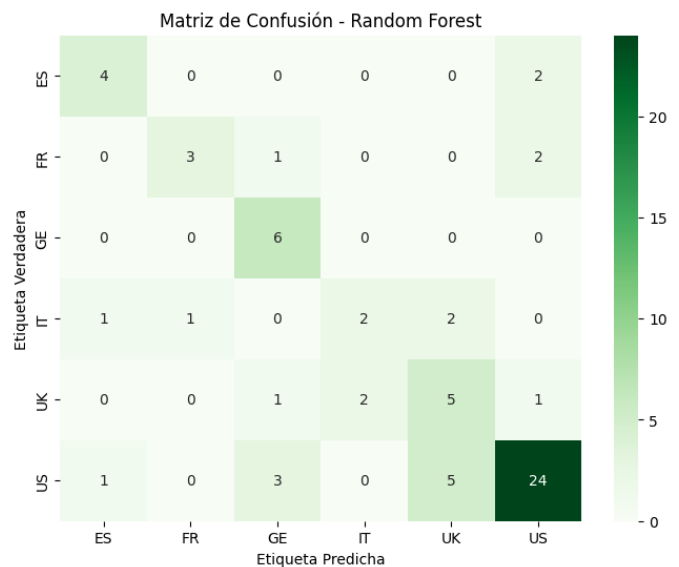


Figura 12. Matriz de confusión para Random Forest en el conjunto de test, permitiendo comparar su patrón de errores con el modelo MLP.

XII. CONCLUSIONES

1. El **MLP Classifier** fue el modelo más efectivo para clasificar acentos en este dataset usando características MFCC, validación cruzada estratificada y SMOTE, logrando el mejor F1-macro promedio y un desempeño aceptable en test.
2. El desbalance de clases sigue afectando, con peor rendimiento en las clases minoritarias pese al uso de SMOTE.
3. Las técnicas de reducción de dimensionalidad (SFS y PCA) no mejoraron el rendimiento; SFS redujo características para MLP pero sin mejora en test, y PCA degradó significativamente el desempeño, indicando que la información clave podría estar distribuida en las características originales.
4. Los resultados son inferiores a los de estudios con arquitecturas deep learning más avanzadas y datasets más grandes, mostrando el potencial de estas técnicas para problemas complejos de clasificación de audio.

En resumen, aunque el MLP optimizado y el manejo del desbalance dieron resultados razonables, la clasificación de acentos es un reto que podría beneficiarse de modelos más sofisticados, mejores características y datasets más balanceados y amplios. La reducción de dimensionalidad con SFS o PCA no fue útil en este caso.

REFERENCIAS

- [1] H. He and E. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, pp. 1263–1284, 9 2009.
- [2] W. Ke, "Study on recognition and classification of english accents using deep learning algorithms," *Journal of Intelligent Systems*, vol. 32, p. 20230174, 2023. Open Access.
- [3] Z. Ma and E. Fokoué, "A comparison of classifiers in performing speaker accent recognition using mfccs," *Open Journal of Statistics*, vol. 04, pp. 258–266, 2014.
- [4] L. M. Arslan and J. H. L. Hansen, "Language accent classification in american english," *Speech Communication*, vol. 18, no. 4, pp. 353–367, 1996.